🐍 Python — Complete Interview Prep Guide

AI Engineer ke liye — Basic se Advanced tak

> **Tera level:** Java dev → AI Engineer | 5.5 YOE **Style:** Har topic — Definition (English + Hinglish) + Deep Example + Interview Q&A **Java analogies:** Jahan relevant ho — Java se compare kiya hai fast understanding ke liye

---

📖 GLOSSARY — Python Ke Unique Words

> *Java se alag concepts — pehle yeh samjho*

Word	Aasan Matlab
Interpreted	Code seedha run hota hai — pehle compile nahi hota (Java = compile → run)
Dynamic typing	Variable ka type runtime pe decide hota hai — pehle declare nahi karna
Duck typing	"Agar bolta hai quack aur chalta hai duck ki tarah — toh duck hai" — type check nahi, behavior check
Mutable	Object change ho sakta hai after creation — list, dict
Immutable	Object change nahi ho sakta — string, tuple, int
Iterable	Jisme se ek ek item nikaal sako — list, dict, string, generator
Iterator	Ek iterable ka "cursor" — next() call pe next item deta hai
Generator	Lazy iterator — sab values ek saath nahi, ek ek karke deta hai (yield se)
Decorator	Function ke upar @something — us function ko wrap karta hai extra behavior add karne ke liye
Context manager	`with` statement ke saath kaam karta hai — setup aur cleanup automatically
Coroutine	Async function — beech mein pause ho sakta hai (await pe)
Event loop	Async program ka traffic controller — decide karta hai kaunsa coroutine kab chale
GIL	Global Interpreter Lock — Python mein ek waqt ek hi thread truly parallel nahi chal sakti
First-class function	Function ko variable mein store kar sako, argument mein pass kar sako
Closure	Inner function jo outer function ke variables "remember" karta hai
Unpacking	`a, b = [1, 2]` — list/tuple ke items ko variables mein split karna
Walrus operator	`:=` — assignment aur expression ek saath
f-string	`f"Hello {name}"` — string ke andar variable directly
Comprehension	Ek line mein list/dict/set banana — `[x*2 for x in range(10)]`

📌 TOPIC 1: Variables aur Data Types

Interview Importance: ⭐ ⭐ ⭐ (Medium) — Basics poochhe jaate hain warmup mein

---

🔷 Variables — Dynamic Typing

Definition (English):

> In Python, variables are dynamically typed — the type is inferred at runtime based on the value assigned, not declared explicitly.

Hinglish:

- Java mein: `String name = "Rahul"` — type pehle batana padta hai
- Python mein: `name = "Rahul"` — Python khud samjhta hai ki yeh string hai
- Variable ka type kisi bhi waqt change ho sakta hai — yeh dynamic typing hai

```python
# ── BASIC VARIABLE ASSIGNMENT ──
name = "Rahul"          # str — Python ne khud detect kiya
age = 25                # int
salary = 85000.50       # float
is_employed = True      # bool — True/False (capital T/F — Java se alag!)
nothing = None          # None — Java ka null

# ── TYPE CHECK ──
print(type(name))       # <class 'str'>
print(type(age))        # <class 'int'>
print(isinstance(age, int))  # True — Java ka instanceof jaisa

# ── DYNAMIC TYPING — type change ho sakta hai ──
x = 10          # x is int
x = "hello"     # ab x is str — Java mein yeh error deta
x = [1, 2, 3]  # ab x is list — Python allow karta hai

# ── MULTIPLE ASSIGNMENT ──
a = b = c = 0   # teeno ek saath 0
x, y, z = 1, 2, 3  # unpacking — Java mein alag alag likhna padta

# ── SWAP — Python ka magic ──
a, b = 10, 20
a, b = b, a     # swap — koi temp variable nahi chahiye!
# Java mein: int temp = a; a = b; b = temp;
```

🔷 Data Types — Sab Important Wale

```python
```

```python
# ═══════════════════════════════════
# 1. STRING — str
# ═══════════════════════════════════
name = "Rahul"
multi_line = """Yeh ek
multi-line string hai"""  # triple quotes

# String methods — commonly used
print(name.upper())         # "RAHUL"
print(name.lower())         # "rahul"
print(name.strip())         # whitespace remove karo dono ends se
print(name.split(","))      # list mein tod do
print(",".join(["a","b"]))  # list ko string mein jodo → "a,b"
print(name.replace("R","r")) # "rahul"
print(name.startswith("Ra")) # True
print(len(name))            # 5

# f-string — most important!
city = "Delhi"
experience = 5.5
intro = f"Main {name} hoon, {city} se, {experience} YOE."
# Output: "Main Rahul hoon, Delhi se, 5.5 YOE."
# f-string ke andar expressions bhi chal sakte hain:
result = f"2 + 2 = {2 + 2}"        # "2 + 2 = 4"
upper = f"{name.upper()} rocks!"   # "RAHUL rocks!"

# String immutable hai — change nahi ho sakti
s = "hello"
# s[0] = "H"  # ← ERROR! String immutable hai
s = "Hello"   # naya string bana — purana "hello" garbage collected

# ═══════════════════════════════════
# 2. INTEGER aur FLOAT — int, float
# ═══════════════════════════════════
x = 10          # int — unlimited size (Java ka BigInteger jaisa automatically)
y = 3.14        # float — 64-bit double precision

# Division — Java se DIFFERENT!
print(10 / 3)   # 3.3333... — float (Java mein int/int = int!)
print(10 // 3)  # 3 — floor division (int result)
print(10 % 3)   # 1 — remainder
print(2 ** 10)  # 1024 — power (Java mein Math.pow())

# Type conversion
x = int("42")       # string → int
y = float("3.14")   # string → float
```

```python
z = str(100)        # int → string

# ═══════════════════════════════════
# 3. BOOLEAN — bool
# ═══════════════════════════════════
is_ready = True    # Capital T — Java se alag (true lowercase nahi)
is_done = False    # Capital F

# Truthy aur Falsy — important concept
# Yeh values False ki tarah behave karti hain:
falsy_values = [
    False, None, 0, 0.0, "", [], {}, set()
]
# Baaki sab True ki tarah behave karte hain

# Practical use:
docs = []
if docs:            # empty list = falsy — yeh kaam karta hai!
    print("Docs mile")
else:
    print("Koi doc nahi")  # ← yeh print hoga

# ═══════════════════════════════════
# 4. LIST — list (Java ka ArrayList)
# ═══════════════════════════════════
# Mutable — change ho sakti hai
# Ordered — order maintain hota hai
# Duplicates allowed

fruits = ["apple", "banana", "cherry"]
mixed = [1, "hello", True, 3.14]  # alag types ek saath — Java se different

# Access
print(fruits[0])    # "apple" — 0 indexed
print(fruits[-1])   # "cherry" — last element (negative indexing!)
print(fruits[1:3])  # ["banana", "cherry"] — slicing

# Common operations
fruits.append("mango")      # end mein add
fruits.insert(1, "grape")   # position pe add
fruits.remove("banana")     # value se remove
popped = fruits.pop()       # last element nikalo aur remove
fruits.sort()               # alphabetical sort (in-place)
length = len(fruits)        # size

# AI Engineer mein kahan use hota hai:
retrieved_docs = []
```

```python
for doc in search_results:
    retrieved_docs.append(doc.content)  # RAG pipeline mein common

# List comprehension — powerful one-liner
doc_contents = [doc.content for doc in search_results]  # same kaam, ek line

# ═══════════════════════════════════
# 5. TUPLE — tuple (immutable list)
# ═══════════════════════════════════
# Immutable — create hone ke baad change nahi ho sakta
# Ordered, duplicates allowed
coordinates = (28.6, 77.2)    # latitude, longitude
rgb = (255, 128, 0)           # parentheses

# Access same as list
print(coordinates[0])   # 28.6

# Tuple unpacking — bahut useful
lat, lng = coordinates  # seedha unpack!
r, g, b = rgb

# Kab use karo:
# Return multiple values from function
def get_user_info():
    return "Rahul", 25, "Delhi"  # yeh tuple return karta hai

name, age, city = get_user_info()  # unpack karo

# ═══════════════════════════════════
# 6. DICTIONARY — dict (Java ka HashMap)
# ═══════════════════════════════════
# Key-value pairs
# Mutable
# Keys unique honi chahiye
# Python 3.7+ mein insertion order maintain hota hai

user = {
    "name": "Rahul",
    "age": 25,
    "skills": ["Python", "Java", "LangGraph"],
    "is_active": True
}

# Access
print(user["name"])              # "Rahul" — KeyError agar key nahi hai
print(user.get("age"))           # 25 — safe access
print(user.get("phone", "N/A"))  # "N/A" — default value agar key nahi
```

```python
# Add/Update
user["city"] = "Delhi"           # naya key add
user["age"] = 26                 # existing update

# Common operations
print(user.keys())    # dict_keys(['name', 'age', 'skills', ...])
print(user.values())  # dict_values(['Rahul', 26, ...])
print(user.items())   # dict_items([('name', 'Rahul'), ...])

# Check key exists
if "email" in user:
    print(user["email"])

# Delete
del user["is_active"]            # key delete karo
popped = user.pop("city", None)  # safe delete with default

# AI Engineer mein — LangGraph State bhi dict hai!
agent_state = {
    "messages": [],
    "query": "RAG kya hai?",
    "docs": [],
    "answer": ""
}

# ═══════════════════════════════════
# 7. SET — set (unique values only)
# ═══════════════════════════════════
# Unordered, no duplicates, mutable
unique_tags = {"python", "ai", "langraph", "python"}  # "python" ek baar
print(unique_tags)  # {'python', 'ai', 'langraph'} — duplicate remove

# Operations
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}
print(set_a | set_b)  # Union: {1, 2, 3, 4, 5, 6}
print(set_a & set_b)  # Intersection: {3, 4}
print(set_a - set_b)  # Difference: {1, 2}

# AI use case: Unique document IDs track karo
processed_doc_ids = set()
for doc in documents:
    if doc.id not in processed_doc_ids:
        process(doc)
        processed_doc_ids.add(doc.id)
```

📌 TOPIC 2: Control Flow

Interview Importance: ⭐ ⭐ (Basic) — Mostly syntax questions

```python
```

```python
# ═══════════════════════════════════
# IF/ELIF/ELSE
# ═══════════════════════════════════
score = 0.85

if score >= 0.9:
    verdict = "Excellent"
elif score >= 0.7:
    verdict = "Good"        # ← yeh chalega
elif score >= 0.5:
    verdict = "Average"
else:
    verdict = "Poor"

# One-liner ternary — Java ka (condition ? a : b) jaisa
verdict = "Pass" if score >= 0.7 else "Fail"

# ═══════════════════════════════════
# FOR LOOP
# ═══════════════════════════════════
# Python mein for = for-each (Java ka enhanced for)
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)

# range() — number sequence
for i in range(5):        # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):     # 1, 2, 3, 4, 5
    print(i)

for i in range(0, 10, 2): # 0, 2, 4, 6, 8 — step 2
    print(i)

# enumerate() — index + value dono chahiye
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
# 0: apple
# 1: banana
# 2: cherry

# zip() — do lists ek saath
names = ["Rahul", "Priya", "Amit"]
scores = [0.9, 0.8, 0.7]
```

```python
for name, score in zip(names, scores):
    print(f"{name}: {score}")

# AI Engineer use case — documents process karna
for i, doc in enumerate(retrieved_docs):
    print(f"Document {i+1}: {doc.content[:100]}...")  # first 100 chars

# ═══════════════════════════════════
# WHILE LOOP
# ═══════════════════════════════════
# LangGraph mein important — agent retry logic
retry_count = 0
max_retries = 3

while retry_count < max_retries:
    try:
        result = call_llm()
        break           # success — loop band karo
    except Exception:
        retry_count += 1
        if retry_count == max_retries:
            raise       # max retries khatam — error raise karo

# break aur continue
for i in range(10):
    if i == 3:
        continue    # 3 skip karo, loop continue
    if i == 7:
        break       # 7 pe loop band karo
    print(i)        # 0, 1, 2, 4, 5, 6
```

📌 TOPIC 3: Functions

Interview Importance: ⭐ ⭐ ⭐ ⭐ (High) — Parameters, return values, scope

---

```python
```

```python
# ═══════════════════════════════════
# BASIC FUNCTION
# ═══════════════════════════════════
def greet(name: str) -> str:
    """
    Docstring — function ka description
    Args:
        name: User ka naam
    Returns:
        Greeting string
    """
    return f"Namaste, {name}!"

result = greet("Rahul")   # "Namaste, Rahul!"

# ═══════════════════════════════════
# DEFAULT PARAMETERS
# ═══════════════════════════════════
def create_llm_config(
    model: str = "gpt-4o",      # default value
    temperature: float = 0.7,    # default value
    max_tokens: int = 1000       # default value
) -> dict:
    return {
        "model": model,
        "temperature": temperature,
        "max_tokens": max_tokens
    }

# Different ways to call:
config1 = create_llm_config()                    # sab defaults
config2 = create_llm_config("gpt-4o-mini")      # model override
config3 = create_llm_config(temperature=0.0)     # sirf temperature override
config4 = create_llm_config("claude", 0.5, 500) # sab override

# ═══════════════════════════════════
# *args — Variable Positional Arguments
# ═══════════════════════════════════
# Java mein: String... args  ya  Object... args

def sum_all(*numbers):
    # numbers ek tuple ban jaata hai andar
    # *args = "jo bhi positional arguments aayein, sab ek tuple mein le lo"
    total = 0
    for n in numbers:
        total += n
```

```python
    return total

print(sum_all(1, 2))          # 3
print(sum_all(1, 2, 3, 4, 5)) # 15
print(sum_all())               # 0

# AI use case:
def process_documents(*docs, chunk_size=500):
    # Multiple docs pass kar sako
    results = []
    for doc in docs:
        chunks = split_into_chunks(doc, chunk_size)
        results.extend(chunks)
    return results

# ═══════════════════════════════════
# **kwargs — Variable Keyword Arguments
# ═══════════════════════════════════
# **kwargs = "jo bhi keyword arguments aayein, sab ek dict mein le lo"

def configure_agent(**settings):
    # settings ek dict ban jaata hai andar
    print(f"Settings received: {settings}")

    model = settings.get("model", "gpt-4o")
    temp = settings.get("temperature", 0.7)
    return {"model": model, "temperature": temp}

configure_agent(model="gpt-4o-mini", temperature=0.0, max_tokens=500)
# settings = {"model": "gpt-4o-mini", "temperature": 0.0, "max_tokens": 500}

# Dono saath — *args aur **kwargs
def flexible_function(*args, **kwargs):
    print(f"Args: {args}")
    print(f"Kwargs: {kwargs}")

flexible_function(1, 2, 3, name="Rahul", city="Delhi")
# Args: (1, 2, 3)
# Kwargs: {'name': 'Rahul', 'city': 'Delhi'}

# Unpacking with * aur **
def add(a, b, c):
    return a + b + c

numbers = [1, 2, 3]
result = add(*numbers)      # list unpack karke pass karo — add(1, 2, 3)
```

```python
config = {"a": 1, "b": 2, "c": 3}
result = add(**config)      # dict unpack karke pass karo — add(a=1, b=2, c=3)

# ═══════════════════════════════════
# MULTIPLE RETURN VALUES
# ═══════════════════════════════════
def evaluate_rag_response(question: str, answer: str, context: str):
    faithfulness = calculate_faithfulness(answer, context)
    relevancy = calculate_relevancy(question, answer)

    # Tuple return karta hai — multiple values
    return faithfulness, relevancy, faithfulness > 0.7

# Unpack karo
faith, rel, passed = evaluate_rag_response(q, a, ctx)

# ═══════════════════════════════════
# FIRST-CLASS FUNCTIONS
# ═══════════════════════════════════
# Python mein functions = objects hain
# Variable mein store kar sako, argument mein pass kar sako

def double(x):
    return x * 2

def triple(x):
    return x * 3

# Function ko variable mein store karo
operation = double
print(operation(5))   # 10

# Function ko argument mein pass karo
def apply_to_list(func, numbers):
    return [func(n) for n in numbers]

result = apply_to_list(double, [1, 2, 3, 4])  # [2, 4, 6, 8]
result = apply_to_list(triple, [1, 2, 3, 4])  # [3, 6, 9, 12]

# AI use case — different evaluation functions
evaluators = [faithfulness_eval, relevancy_eval, correctness_eval]
scores = [evaluator(answer, context) for evaluator in evaluators]

# ═══════════════════════════════════
# LAMBDA FUNCTIONS
# ═══════════════════════════════════
# Anonymous function — ek line mein
```

```python
# Java ka lambda expression jaisa: x -> x * 2

# Normal function:
def square(x):
    return x * x

# Lambda equivalent:
square = lambda x: x * x
print(square(5))   # 25

# Lambda with multiple args:
add = lambda x, y: x + y
print(add(3, 4))   # 7

# Practical use — sort with custom key
docs = [
    {"content": "short", "score": 0.9},
    {"content": "medium length", "score": 0.7},
    {"content": "x", "score": 0.95}
]

# Score ke hisaab se sort karo (descending)
sorted_docs = sorted(docs, key=lambda d: d["score"], reverse=True)
# key= ek function accept karta hai — lambda perfect hai

# LCEL mein lambda:
from langchain_core.runnables import RunnableLambda
format_docs = RunnableLambda(lambda docs: "\n\n".join([d.page_content for d in docs]

# ═══════════════════════════════
# SCOPE — Local vs Global
# ═══════════════════════════════
count = 0  # global variable

def increment():
    global count    # global keyword — bahar wala count modify karo
    count += 1      # bina global ke error aata

def get_count():
    return count    # read karna — global keyword ki zaroorat nahi

# Closure — inner function outer ke variables yaad rakhta hai
def make_counter(start=0):
    count = start  # outer function ka variable

    def counter():
        nonlocal count  # outer scope ka variable modify karo
```

```
        count += 1
        return count

    return counter  # function return karo!

my_counter = make_counter(10)
print(my_counter())   # 11
print(my_counter())   # 12
print(my_counter())   # 13
# count = 13 yaad hai — closure!
```

Interview Questions — Functions:

- *"*args aur *kwargs mein kya fark hai?"* — args tuple, kwargs dict; positional vs keyword
- *"Lambda function kab use karte ho?"* — one-liner, sort key, LCEL chains mein
- *"Closure kya hota hai?"* — inner function outer ke variables yaad rakhta hai

---

📌 TOPIC 4: List Comprehensions (aur dict, set)

Interview Importance: ⭐ ⭐ ⭐ ⭐ (High) — Code reading mein aata hai, elegant Python

---

🔷 List Comprehension

Definition (English):

> List comprehension is a concise way to create lists by applying an expression to each item in an iterable, optionally filtering with a condition — all in a single line.

Hinglish:

- `for` loop se list banane ka shortcut — ek line mein
- Java ka `stream().map().filter().collect()` jaisa — but simpler

```python
```

```python
# ═══════════════════════════════════
# BASIC — Without vs With Comprehension
# ═══════════════════════════════════

# OLD WAY — for loop se
squares = []
for i in range(10):
    squares.append(i * i)
# squares = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# NEW WAY — list comprehension
squares = [i * i for i in range(10)]
# Same result, ek line!

# FORMULA: [expression FOR item IN iterable]
#           ↑what to do  ↑for each  ↑source

# ═══════════════════════════════════
# WITH CONDITION — filter bhi karo
# ═══════════════════════════════════

# FORMULA: [expression FOR item IN iterable IF condition]

# Even numbers sirf
evens = [i for i in range(20) if i % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# High score documents sirf
all_docs = [
    {"content": "LangGraph kya hai", "score": 0.9},
    {"content": "Python basic", "score": 0.4},
    {"content": "RAG pipeline", "score": 0.8},
    {"content": "Java tutorial", "score": 0.3},
]

# Score > 0.7 wale documents sirf
relevant_docs = [doc for doc in all_docs if doc["score"] > 0.7]
# [{"content": "LangGraph kya hai", "score": 0.9},
#  {"content": "RAG pipeline", "score": 0.8}]

# ═══════════════════════════════════
# AI ENGINEER — Real Use Cases
# ═══════════════════════════════════

# Use case 1: Document contents extract karo
doc_texts = [doc.page_content for doc in retrieved_docs]
```

```python
# Equivalent to:
# doc_texts = []
# for doc in retrieved_docs:
#     doc_texts.append(doc.page_content)

# Use case 2: Messages se sirf human messages nikalo
from langchain_core.messages import HumanMessage
human_msgs = [
    msg.content
    for msg in state["messages"]
    if isinstance(msg, HumanMessage)
]

# Use case 3: Scores normalize karo
scores = [0.9, 0.7, 0.5, 0.8]
normalized = [round(s * 100, 1) for s in scores]
# [90.0, 70.0, 50.0, 80.0]

# Use case 4: Tool names extract karo from tool calls
tool_names = [
    call["name"]
    for call in ai_message.tool_calls
    if "name" in call
]

# ═══════════════════════════════════
# NESTED COMPREHENSION
# ═══════════════════════════════════

# 2D list flatten karna
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row]
# [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Documents ke andar sentences nikalo
all_sentences = [
    sentence
    for doc in documents
    for sentence in doc.content.split(".")
    if len(sentence.strip()) > 10  # short fragments avoid karo
]

# ═══════════════════════════════════
# DICT COMPREHENSION
# ═══════════════════════════════════

# FORMULA: {key: value FOR item IN iterable}
```

```
# List ko dict mein convert karo
words = ["apple", "banana", "cherry"]
word_lengths = {word: len(word) for word in words}
# {"apple": 5, "banana": 6, "cherry": 6}

# Existing dict transform karo
old_config = {"model": "gpt-4", "temp": 0.7, "tokens": 1000}
new_config = {k.upper(): v for k, v in old_config.items()}
# {"MODEL": "gpt-4", "TEMP": 0.7, "TOKENS": 1000}

# AI use case: Evaluation scores dict banana
questions = ["Q1", "Q2", "Q3"]
scores = [0.9, 0.8, 0.7]
eval_results = {q: s for q, s in zip(questions, scores)}
# {"Q1": 0.9, "Q2": 0.8, "Q3": 0.7}

# ═══════════════════════════════════
# SET COMPREHENSION
# ═══════════════════════════════════

# FORMULA: {expression FOR item IN iterable}
unique_words = {word.lower() for sentence in sentences for word in sentence.split()}
```

Interview Questions:

- *"List comprehension kya hai? Normal loop se better kyun?"* — concise, readable, slightly faster
- *"Yeh code kya karega:* `[x for x in range(10) if x % 2 == 0]`*?"* — even numbers: [0,2,4,6,8]
- *"Dict comprehension ka example do"* — `{k: v for k, v in dict.items()}`

📌 TOPIC 5: Decorators ⭐ (AI Engineer Must Know)

Interview Importance: ⭐ ⭐ ⭐ ⭐ ⭐ (Critical) — @tool, @property, FastAPI @app.get() — har jagah

---

🔷 Decorator Kya Hota Hai

Definition (English):

> A decorator is a function that takes another function as input, adds extra behavior to it (before and/or after execution), and returns the modified function — using the `@` syntax as

> syntactic sugar.

Hinglish:

- Ek function jo doosre function ko "wrap" karta hai — extra kaam karne ke liye
- Java ka `@Annotation` jaisa dikhta hai — but behavior alag hai
- `@tool` decorator = LangChain ko bolta hai "yeh function ek tool hai LLM ke liye"
- `@app.get("/")` = FastAPI ko bolta hai "yeh function GET request handle karega"

```python
```

```python
# ═══════════════════════════════════
# DECORATOR ANDAR SE KAISE KAAM KARTA HAI
# ═══════════════════════════════════

# Step 1: Bina decorator — manually wrap karna
def my_function():
    print("Main kaam kar raha hoon")

def logger(func):
    """
    Yeh wrapper function hai — func ko wrap karta hai
    logging add karke
    """
    def wrapper(*args, **kwargs):
        print(f"▶ {func.__name__} shuru hua")  # before
        result = func(*args, **kwargs)           # original function call
        print(f"✓ {func.__name__} khatam hua")  # after
        return result
    return wrapper  # wrapper function return karo

# Manually wrap karna:
my_function = logger(my_function)
my_function()
# ▶ my_function shuru hua
# Main kaam kar raha hoon
# ✓ my_function khatam hua

# Step 2: @ syntax — shortcut hai
# @logger  ←  yeh exactly `my_function = logger(my_function)` hai!

@logger
def my_function():
    print("Main kaam kar raha hoon")

my_function()   # same output — @ ne wrap kar diya automatically

# ═══════════════════════════════════
# TIMING DECORATOR — AI ENGINEER USE CASE
# ═══════════════════════════════════
import time
from functools import wraps

def timer(func):
    """Function execution time measure karo"""
    @wraps(func)  # ← important! Original function ka naam/docstring preserve karo
    def wrapper(*args, **kwargs):
```

```python
        start = time.time()                          # start time
        result = func(*args, **kwargs)               # original function
        end = time.time()                            # end time
        duration = (end - start) * 1000             # milliseconds
        print(f"⏱ {func.__name__} took {duration:.2f}ms")
        return result
    return wrapper

@timer
def call_llm(prompt: str) -> str:
    """LLM API call simulate karo"""
    time.sleep(0.5)  # simulate latency
    return "LangGraph ek framework hai..."

response = call_llm("RAG kya hai?")
# ⏱ call_llm took 500.23ms

# ═══════════════════════════════════
# RETRY DECORATOR
# ═══════════════════════════════════
def retry(max_attempts: int = 3, delay: float = 1.0):
    """
    Decorator with arguments — ek aur layer ka wrapper chahiye
    @retry(max_attempts=3) ka matlab hai:
    retry(3) call hota hai → ek decorator return karta hai
    us decorator se function wrap hota hai
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_error = None
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)  # try karo
                except Exception as e:
                    last_error = e
                    print(f"Attempt {attempt + 1} failed: {e}")
                    if attempt < max_attempts - 1:
                        time.sleep(delay * (2 ** attempt))  # exponential backoff
            raise last_error  # sab attempts fail — raise karo
        return wrapper
    return decorator

@retry(max_attempts=3, delay=1.0)
def fragile_api_call():
    """Kabhi kabhi fail hone wala API call"""
    import random
```

```python
    if random.random() < 0.7:  # 70% fail hoga
        raise ConnectionError("API unreachable")
    return "Success!"

result = fragile_api_call()  # automatically retry karega

# ═══════════════════════════════════
# @property DECORATOR
# ═══════════════════════════════════
# Java ka getter/setter jaisa — but cleaner

class EmbeddingModel:
    def __init__(self, model_name: str, dimensions: int):
        self._model_name = model_name      # _ prefix = private convention
        self._dimensions = dimensions
        self._cost_per_token = 0.0001

    @property
    def model_name(self):
        """Getter — model_name access karo"""
        return self._model_name

    @property
    def dimensions(self):
        return self._dimensions

    @property
    def monthly_cost_estimate(self):
        """Computed property — har baar fresh calculate"""
        avg_tokens_per_day = 100000
        return avg_tokens_per_day * 30 * self._cost_per_token

    @model_name.setter
    def model_name(self, value: str):
        """Setter — validation ke saath"""
        valid_models = ["text-embedding-ada-002", "text-embedding-3-small"]
        if value not in valid_models:
            raise ValueError(f"Invalid model: {value}")
        self._model_name = value

# Use karo — method call nahi, attribute access jaisa lagta hai!
model = EmbeddingModel("text-embedding-3-small", 1536)
print(model.model_name)           # "text-embedding-3-small" — getter call hua
print(model.dimensions)           # 1536
print(model.monthly_cost_estimate) # 300.0 — auto calculate

model.model_name = "text-embedding-ada-002"  # setter call hua
```

```python
model.model_name = "invalid-model"  # ← ValueError!

# ═══════════════════════════════════
# @staticmethod aur @classmethod
# ═══════════════════════════════════
class VectorDB:
    _instance = None  # Singleton pattern ke liye

    def __init__(self, url: str, collection: str):
        self.url = url
        self.collection = collection
        self._connected = False

    @staticmethod
    def validate_url(url: str) -> bool:
        """
        @staticmethod = class ka method hai but self nahi chahiye
        Class ya object — dono se call kar sako
        Java ka static method jaisa
        """
        return url.startswith("http://") or url.startswith("https://")

    @classmethod
    def create_local(cls, collection: str):
        """
        @classmethod = class method — cls (class itself) milta hai, self nahi
        Factory method pattern
        Java ka static factory method jaisa
        """
        return cls("http://localhost:6333", collection)

    @classmethod
    def get_instance(cls, url: str, collection: str):
        """Singleton pattern"""
        if cls._instance is None:
            cls._instance = cls(url, collection)
        return cls._instance

# Use karo
print(VectorDB.validate_url("https://pinecone.io"))  # True — class se call
local_db = VectorDB.create_local("my_docs")          # factory method
db = VectorDB.get_instance("https://pinecone.io", "prod")

# ═══════════════════════════════════
# LANGCHAIN @tool DECORATOR
# ═══════════════════════════════════
from langchain_core.tools import tool
```

```python
@tool
def search_documentation(query: str) -> str:
    """
    Search internal documentation for relevant information.
    Use this when you need to find specific technical details.

    Args:
        query: The search query string

    Returns:
        Relevant documentation snippets
    """
    # Yeh decorator yeh karta hai:
    # 1. Function ka naam → tool ka naam: "search_documentation"
    # 2. Docstring → tool ka description (LLM yeh padhta hai!)
    # 3. Type hints → JSON schema generate karta hai
    # 4. Function ko Tool object mein wrap karta hai
    docs = vector_db.search(query, k=3)
    return "\n".join([d.page_content for d in docs])

# Ab yeh tool LLM use kar sakta hai
print(search_documentation.name)        # "search_documentation"
print(search_documentation.description) # docstring
print(search_documentation.args_schema) # Pydantic schema
```

Interview Questions — Decorators:

- *"Decorator kya hota hai? Andar se kaise kaam karta hai?"* — function jo function wrap kare, @syntax shortcut hai
- *"@wraps kyun use karte ho?"* — original function ka naam aur docstring preserve karne ke liye
- *"@tool decorator kya karta hai LangChain mein?"* — function ko Tool object mein convert, docstring = LLM description, type hints = JSON schema
- *"@property vs regular method?"* — property = attribute jaisa access, method = function call syntax

---

📌 TOPIC 6: Generators aur yield ⭐ (LLM Streaming)

Interview Importance: ⭐ ⭐ ⭐ ⭐ ⭐ (Critical) — LLM streaming response ke liye must know

---

🔷 Generator Kya Hota Hai

Definition (English):

> A generator is a special function that uses `yield` instead of `return` — it produces values one at a time (lazily), pausing execution between each value, without storing all values in memory at once.

Hinglish:

- Normal function = ek baar mein sab kuch return karta hai
- Generator = thoda thoda karta hai — ek value do, pause, phir next value
- `return` ki jagah `yield` use hota hai
- LLM streaming = generator — tokens ek ek karke aate hain, sab ek saath nahi

```python
```

```python
# ═══════════════════════════════════
# NORMAL FUNCTION vs GENERATOR
# ═══════════════════════════════════

# Normal function — sab ek saath return
def get_all_squares(n: int) -> list:
    result = []
    for i in range(n):
        result.append(i * i)
    return result  # sab values memory mein hain

squares = get_all_squares(1000000)  # 1 million numbers — RAM mein!
# Problem: Agar n = 10 million? Out of memory!

# Generator — ek ek karke
def square_generator(n: int):
    for i in range(n):
        yield i * i  # yield = value do aur PAUSE karo
        # Next next() call pe yahan se continue hoga

# Use karo
gen = square_generator(1000000)  # Sirf generator object bana — no memory usage!
print(next(gen))  # 0 — first value
print(next(gen))  # 1 — second value
print(next(gen))  # 4 — third value

# for loop ke saath — automatically next() call karta hai
for square in square_generator(10):
    print(square)  # 0, 1, 4, 9, 16, 25, 36, 49, 64, 81

# ═══════════════════════════════════
# YIELD KAISE KAAM KARTA HAI — Step by Step
# ═══════════════════════════════════

def countdown(n: int):
    print(f"Countdown shuru: {n} se")
    while n > 0:
        yield n        # ← yahan function PAUSE ho jaata hai
        # Jab next next() call hoga, yahan se continue hoga
        n -= 1
    print("Countdown khatam!")

counter = countdown(3)

print("Generator banaya — abhi kuch nahi chala!")
print(next(counter))  # "Countdown shuru: 3 se" print hoga, phir 3 yield
```

```python
print(next(counter))  # 2 yield
print(next(counter))  # 1 yield
# print(next(counter))  # StopIteration exception — generator khatam

# ═══════════════════════════════
# LLM STREAMING — Real Use Case
# ═══════════════════════════════

# LLM tokens ek ek karke generate karta hai
# Streaming = user ko wait nahi karwana — tokens aate jao, dikhate jao

# Generator function for streaming
def stream_llm_response(prompt: str):
    """
    LLM se streaming response lao
    Har token aate hi yield karo — user ko immediately dikhe
    """
    # LLM streaming API call
    for chunk in llm.stream(prompt):
        if chunk.content:  # empty chunks skip karo
            yield chunk.content  # ek ek token yield karo

# Use karo — token by token print
for token in stream_llm_response("RAG kya hai?"):
    print(token, end="", flush=True)  # ek ek token print, newline nahi
# Output: "R-A-G ek technique hai jo..." — streaming effect!

# FastAPI ke saath — StreamingResponse
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.post("/chat/stream")
async def stream_chat(query: str):
    def generate():
        for chunk in llm.stream(query):
            if chunk.content:
                # Server-Sent Events format
                yield f"data: {chunk.content}\n\n"
        yield "data: [DONE]\n\n"  # stream khatam

    return StreamingResponse(
        generate(),
        media_type="text/event-stream"
    )
```

```python
# ═══════════════════════════════════
# ASYNC GENERATOR — FastAPI + LangGraph
# ═══════════════════════════════════

async def async_stream_response(query: str):
    """
    Async generator — async for ke saath use karo
    FastAPI async endpoints ke liye perfect
    """
    async for chunk in llm.astream(query):
        if chunk.content:
            yield chunk.content

@app.post("/chat/async-stream")
async def async_stream_chat(query: str):
    async def generate():
        async for token in async_stream_response(query):
            yield f"data: {token}\n\n"

    return StreamingResponse(generate(), media_type="text/event-stream")

# LangGraph streaming
@app.post("/agent/stream")
async def stream_agent(query: str):
    async def generate():
        async for event in langgraph_app.astream(
            {"messages": [HumanMessage(content=query)]},
            config={"configurable": {"thread_id": "session_1"}}
        ):
            for node_name, node_output in event.items():
                if "messages" in node_output:
                    last_msg = node_output["messages"][-1]
                    if hasattr(last_msg, "content") and last_msg.content:
                        yield f"data: {last_msg.content}\n\n"

    return StreamingResponse(generate(), media_type="text/event-stream")

# ═══════════════════════════════════
# GENERATOR EXPRESSIONS
# ═══════════════════════════════════
# List comprehension jaisa — but lazy (generator)
# () use karo [] ki jagah

# List comprehension — sab memory mein
squares_list = [x * x for x in range(1000000)]  # 1M items memory mein

# Generator expression — lazy
```

```python
squares_gen = (x * x for x in range(1000000))   # Sirf generator object

# sum() generator ke saath — efficient
total = sum(x * x for x in range(1000000))  # memory efficient!

# AI use case — large document processing
total_tokens = sum(
    len(doc.page_content.split())  # har doc ke words count karo
    for doc in all_documents        # lazily process karo
    if doc.score > 0.7              # relevant sirf
)
```

Interview Questions — Generators:

- *"Generator aur normal function mein kya fark hai?"* — yield vs return, lazy vs eager, memory efficient
- *"LLM streaming kaise implement karte ho?"* — generator function yield karta hai chunks, FastAPI StreamingResponse use karo
- *"Generator expression kab prefer karte ho list pe?"* — large data, memory concern, sirf ek baar iterate karna ho

---

📌 TOPIC 7: Context Managers ⭐ (LangSmith, DB Connections)

Interview Importance: ⭐ ⭐ ⭐ ⭐ (High) — LangSmith tracing, DB, file handling

---

🔷 Context Manager Kya Hota Hai

Definition (English):

> A context manager is an object that defines `__enter__` and `__exit__` methods, used with the `with` statement to automatically handle setup and teardown — ensuring resources are properly released even if an error occurs.

Hinglish:

- `with` statement ke saath kaam karta hai
- Setup automatically hota hai (enter) aur cleanup automatically hota hai (exit)
- Error aaye ya na aaye — cleanup guaranteed
- Java ka `try-with-resources` bilkul same concept hai

```python
```

```python
# ═══════════════════════════════════
# PROBLEM WITHOUT CONTEXT MANAGER
# ═══════════════════════════════════

# File manually open/close karna — risky!
file = open("docs.txt", "r")
try:
    content = file.read()
    # ... process content
finally:
    file.close()  # close karna zaroor — warna resource leak
    # Agar try mein error aaya → finally chalega → close hoga

# WITH CONTEXT MANAGER — cleaner!
with open("docs.txt", "r") as file:
    content = file.read()
    # ... process
# File automatically close ho gayi — with block khatam hone pe
# Error aaye tab bhi close hogi — guaranteed!

# ═══════════════════════════════════
# APNA CONTEXT MANAGER BANANA
# ═══════════════════════════════════

# Method 1: Class ke saath __enter__ aur __exit__
class LLMCallTracer:
    """
    LLM call trace karo — shuru aur khatam dono log karo
    """
    def __init__(self, call_name: str):
        self.call_name = call_name
        self.start_time = None
        self.tokens_used = 0

    def __enter__(self):
        """with block shuru hone pe yeh chalega"""
        import time
        self.start_time = time.time()
        print(f"▶ LLM Call shuru: {self.call_name}")
        return self  # 'as' clause mein yeh milega

    def __exit__(self, exc_type, exc_val, exc_tb):
        """
        with block khatam hone pe yeh chalega
        exc_type: Exception ka type (None agar koi error nahi)
        exc_val: Exception value
```

```
        exc_tb: Traceback
        """
        import time
        duration = (time.time() - self.start_time) * 1000

        if exc_type is not None:
            # Error aaya!
            print(f"× LLM Call failed: {self.call_name} | Error: {exc_val}")
        else:
            print(f"✓ LLM Call done: {self.call_name} | {duration:.0f}ms")

        return False  # False = exception re-raise karo (suppress mat karo)

# Use karo
with LLMCallTracer("RAG Query") as tracer:
    response = llm.invoke("RAG kya hai?")
    tracer.tokens_used = 150
# ▶ LLM Call shuru: RAG Query
# ... LLM call hoti hai ...
# ✓ LLM Call done: RAG Query | 1250ms

# ═══════════════════════════════════
# Method 2: @contextmanager decorator — simpler!
# ═══════════════════════════════════
from contextlib import contextmanager

@contextmanager
def managed_vector_db_connection(url: str, collection: str):
    """
    Vector DB connection manage karo
    yield se pehle = setup (__enter__)
    yield ke baad = cleanup (__exit__)
    """
    print(f"Connecting to vector DB: {url}")
    db = VectorDB(url, collection)
    db.connect()

    try:
        yield db  # ← yeh 'as' clause mein milega
        # with block ka code yahan chalega
    except Exception as e:
        db.rollback()  # error pe rollback
        raise          # exception re-raise karo
    finally:
        db.disconnect()  # error ho ya na ho — disconnect guarantee
        print("DB connection closed")
```

```python
# Use karo
with managed_vector_db_connection("localhost:6333", "docs") as db:
    results = db.search("RAG pipeline", k=5)
    # with block khatam → automatically disconnect

# ═══════════════════════════════════
# MULTIPLE CONTEXT MANAGERS
# ═══════════════════════════════════

# Purana tarika — nested
with open("input.txt") as infile:
    with open("output.txt", "w") as outfile:
        outfile.write(infile.read())

# Naya tarika — ek line!
with open("input.txt") as infile, open("output.txt", "w") as outfile:
    outfile.write(infile.read())

# ═══════════════════════════════════
# LANGSMITH TRACING — Context Manager
# ═══════════════════════════════════
from langchain_core.tracers.context import tracing_v2_enabled

# Specific operation ke liye tracing enable karo
with tracing_v2_enabled(project_name="rag-evaluation"):
    # Yeh block ke andar sab kuch LangSmith mein trace hoga
    result = rag_chain.invoke({"question": "RAG kya hai?"})
    # Trace automatically LangSmith pe jayega

# OpenAI cost tracking
from langchain_community.callbacks import get_openai_callback

with get_openai_callback() as cb:
    result = llm.invoke("LangGraph explain karo")
    print(f"Tokens used: {cb.total_tokens}")
    print(f"Cost: ${cb.total_cost:.4f}")
# Tokens used: 450
# Cost: $0.0045

# MCP Client — context manager
from langchain_mcp_adapters.client import MultiServerMCPClient

async def use_mcp_tools():
    async with MultiServerMCPClient({"calculator": {...}}) as client:
        tools = client.get_tools()
        # Context manager ensure karta hai client properly close ho
```

Interview Questions — Context Managers:

- *"Context manager kya hota hai? Kyun use karte ho?"* — setup/cleanup automatic, resource leak avoid, error hone pe bhi cleanup guaranteed
- *"**enter** aur **exit** kya karte hain?"* — enter: setup aur resource return, exit: cleanup, exception handle
- *"LLM applications mein kahan use kiya?"* — LangSmith tracing, DB connections, OpenAI cost tracking, MCP client

---

📌 TOPIC 8: Exception Handling

Interview Importance: ⭐ ⭐ ⭐ ⭐ (High) — Agent error handling mein critical

---

```python
```

```python
# ═══════════════════════════════════
# BASIC try/except/else/finally
# ═══════════════════════════════════

def safe_llm_call(prompt: str) -> str:
    try:
        # Yeh try karo
        response = llm.invoke(prompt)
        return response.content

    except RateLimitError as e:
        # Specific error handle karo — rate limit
        print(f"Rate limit hit: {e}")
        time.sleep(2)  # wait karo
        return safe_llm_call(prompt)  # retry

    except APITimeoutError as e:
        # Specific error — timeout
        print(f"Timeout: {e}")
        raise  # re-raise karo — caller handle kare

    except (ConnectionError, NetworkError) as e:
        # Multiple exceptions ek saath
        print(f"Network issue: {e}")
        return "Service temporarily unavailable"

    except Exception as e:
        # Sab errors ka fallback
        print(f"Unexpected error: {type(e).__name__}: {e}")
        raise  # unknown errors always re-raise karo

    else:
        # Koi exception nahi aaya toh yeh chalega
        print("LLM call successful!")  # try block ke baad, sirf success pe

    finally:
        # HAMESHA chalega — error ho ya na ho
        # Cleanup code yahan
        log_api_call(prompt)  # hamesha log karo

# ═══════════════════════════════════
# CUSTOM EXCEPTIONS
# ═══════════════════════════════════
class RAGException(Exception):
    """Base class for RAG-related errors"""
    pass
```

```python
class DocumentNotFoundError(RAGException):
    """Koi relevant document nahi mila"""
    def __init__(self, query: str, threshold: float):
        self.query = query
        self.threshold = threshold
        super().__init__(
            f"No documents found for query '{query}' "
            f"with similarity threshold {threshold}"
        )

class HallucinationDetectedError(RAGException):
    """LLM ne hallucinate kiya — faithfulness low hai"""
    def __init__(self, faithfulness_score: float):
        self.score = faithfulness_score
        super().__init__(
            f"Hallucination detected! Faithfulness score: {faithfulness_score}"
        )

class ContextWindowExceededError(RAGException):
    """Token limit exceed ho gayi"""
    pass

# Use karo
def retrieve_and_validate(query: str) -> list:
    docs = vector_db.search(query, k=5)

    if not docs:
        raise DocumentNotFoundError(query, threshold=0.7)

    # Faithfulness check
    if faithfulness_score < 0.5:
        raise HallucinationDetectedError(faithfulness_score)

    return docs

# Handle karo
try:
    docs = retrieve_and_validate("obscure question")
except DocumentNotFoundError as e:
    print(f"Query: {e.query}, Threshold: {e.threshold}")
    # Fallback: broader search try karo
except HallucinationDetectedError as e:
    print(f"Quality too low: {e.score}")
    # Fallback: refine query, retry
except RAGException as e:
    # Sab RAG errors ka generic handler
```

```python
    print(f"RAG failed: {e}")

# ═══════════════════════════════════
# EXCEPTION CHAINING
# ═══════════════════════════════════
try:
    raw_data = fetch_from_db(user_id)
except DatabaseError as e:
    raise UserDataError(f"User {user_id} data unavailable") from e
    # "from e" = original error preserve karo traceback mein

# ═══════════════════════════════════
# LANGGRAPH MEIN ERROR HANDLING PATTERN
# ═══════════════════════════════════
from typing import Optional

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
    error: Optional[str]          # None agar koi error nahi
    error_type: Optional[str]     # error ki category
    retry_count: int

def safe_retrieval_node(state: AgentState) -> dict:
    """Error handling ke saath retrieval"""
    try:
        docs = vector_db.search(state["query"], k=5)
        return {"docs": docs, "error": None}

    except ConnectionError as e:
        return {
            "docs": [],
            "error": str(e),
            "error_type": "CONNECTION_ERROR"
        }
    except TimeoutError as e:
        return {
            "docs": [],
            "error": str(e),
            "error_type": "TIMEOUT",
            "retry_count": state.get("retry_count", 0) + 1
        }
    except Exception as e:
        return {
            "docs": [],
            "error": f"Unexpected: {str(e)}",
```

```
        "error_type": "UNKNOWN"
    }
```

📌 TOPIC 9: Async/Await — Deep Understanding ⭐

Interview Importance: ⭐ ⭐ ⭐ ⭐ ⭐ (Critical) — FastAPI, LangGraph ainvoke, production

---

🔷 Async Kyun Chahiye

Definition (English):

> Asynchronous programming allows a program to handle multiple operations concurrently without blocking — when one operation is waiting (for I/O, API call, DB query), the program can work on other tasks instead of sitting idle.

Hinglish:

- Sync = ek kaam karo, khatam hone ka wait karo, phir aagla kaam
- Async = ek kaam shuru karo, wait karne ki jagah doosra kaam shuru karo
- LLM API call = 2 seconds lagti hai — sync mein poora thread block
- Async mein = LLM call bhejo, wait karne ki jagah aur requests handle karo

```python
```

```python
# ═══════════════════════════════════
# SYNC vs ASYNC — Time Difference
# ═══════════════════════════════════
import time
import asyncio

# SYNC — sequential, slow
def fetch_data_sync(url: str) -> str:
    time.sleep(2)  # network call simulate karo
    return f"Data from {url}"

def sync_main():
    start = time.time()

    # Yeh sequential chalega — ek ke baad ek
    result1 = fetch_data_sync("api1")  # 2 second wait
    result2 = fetch_data_sync("api2")  # 2 second wait
    result3 = fetch_data_sync("api3")  # 2 second wait

    print(f"Total time: {time.time() - start:.1f}s")  # ~6 seconds!

# ASYNC — concurrent, fast
async def fetch_data_async(url: str) -> str:
    await asyncio.sleep(2)  # await = "yahan wait karo but block mat karo"
    return f"Data from {url}"

async def async_main():
    start = time.time()

    # Teeno simultaneously start karo!
    results = await asyncio.gather(
        fetch_data_async("api1"),
        fetch_data_async("api2"),
        fetch_data_async("api3")
    )

    print(f"Total time: {time.time() - start:.1f}s")  # ~2 seconds!

asyncio.run(async_main())

# ═══════════════════════════════════
# ASYNC FUNCTION — COROUTINE
# ═══════════════════════════════════

# async def = coroutine function define karna
async def call_llm_async(prompt: str) -> str:
```

```python
    """
    Async LLM call
    await pe function pause ho jaata hai
    Event loop doosra kaam karta hai pause ke dauran
    """
    # await = "is kaam ka wait karo — but thread block mat karo"
    response = await llm.ainvoke(prompt)
    return response.content

# Async function ko run karne ke tarike:
# 1. asyncio.run() — top level
result = asyncio.run(call_llm_async("Hello"))

# 2. await — doosre async function ke andar
async def process_query(query: str) -> dict:
    answer = await call_llm_async(query)
    return {"query": query, "answer": answer}

# 3. FastAPI automatically handle karta hai
@app.post("/query")
async def query_endpoint(query: str):  # async def — FastAPI ko pata hai
    answer = await call_llm_async(query)
    return {"answer": answer}

# ═══════════════════════════════════
# ASYNCIO.GATHER — Parallel Execution
# ═══════════════════════════════════
async def parallel_llm_calls():
    """
    Multiple LLM calls simultaneously — timing improve karo
    """
    questions = [
        "RAG kya hai?",
        "LangGraph explain karo",
        "Embeddings samjhao"
    ]

    # Teeno calls ek saath!
    # Sequential = 3 * 2s = 6s
    # Parallel = max(2s, 2s, 2s) = 2s
    answers = await asyncio.gather(
        *[call_llm_async(q) for q in questions]
    )

    return dict(zip(questions, answers))

# Error handling with gather
```

```python
async def safe_parallel_calls():
    results = await asyncio.gather(
        call_llm_async("Q1"),
        call_llm_async("Q2"),
        call_llm_async("Q3"),
        return_exceptions=True  # ← errors as values — crash nahi hoga
    )

    for i, result in enumerate(results):
        if isinstance(result, Exception):
            print(f"Call {i} failed: {result}")
        else:
            print(f"Call {i}: {result}")

# ═══════════════════════════════════
# FASTAPI ASYNC ENDPOINT — Complete Example
# ═══════════════════════════════════
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class QueryRequest(BaseModel):
    question: str
    session_id: str = "default"

class QueryResponse(BaseModel):
    answer: str
    sources: list[str]
    latency_ms: float

@app.post("/rag/query", response_model=QueryResponse)
async def rag_query(request: QueryRequest):
    """
    Async RAG endpoint
    Multiple I/O operations — sab async
    """
    import time
    start = time.time()

    try:
        # Step 1: Async embedding
        query_embedding = await embedding_model.aembed_query(request.question)

        # Step 2: Async vector search
        docs = await vector_db.asearch(query_embedding, k=5)
```

```
        if not docs:
            raise HTTPException(
                status_code=404,
                detail="No relevant documents found"
            )

        # Step 3: Async LLM call
        context = "\n\n".join([d.page_content for d in docs])
        answer = await call_llm_async(
            f"Context: {context}\n\nQuestion: {request.question}"
        )

        latency = (time.time() - start) * 1000

        return QueryResponse(
            answer=answer,
            sources=[d.metadata.get("source", "Unknown") for d in docs],
            latency_ms=round(latency, 2)
        )

    except HTTPException:
        raise  # re-raise HTTP exceptions as-is
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# ═══════════════════════════════════
# EVENT LOOP — Concept Samjho
# ═══════════════════════════════════
"""
Event Loop = Traffic controller of async Python

Jaise ek restaurant mein:
- Waiter (Event Loop) multiple tables serve karta hai
- Table 1 ne order diya → waiter kitchen ko bheja (I/O start)
- Waiter wait nahi karta — Table 2 ka order leta hai
- Kitchen ready → waiter Table 1 ko serve karta hai
- Simultaneously multiple tables = high throughput!

Python Event Loop:
- Coroutines ko manage karta hai
- I/O wait pe dusra coroutine chalata hai
- GIL se matter nahi karta — I/O bound tasks ke liye perfect
- CPU bound tasks ke liye multiprocessing use karo
"""

# ═══════════════════════════════════
# ASYNC LANGGRAPH
```

```python
# ═══════════════════════════════════

# Async node — async functions LangGraph mein
async def async_retrieve_node(state: AgentState) -> dict:
    # await = yahan wait karo but block mat karo
    docs = await async_retriever.ainvoke(state["query"])
    return {"docs": docs}

# Async invoke
async def run_agent(query: str) -> dict:
    result = await langgraph_app.ainvoke(
        {"query": query, "messages": []},
        config={"configurable": {"thread_id": "session_1"}}
    )
    return result

# Async streaming
async def stream_agent_response(query: str):
    async for event in langgraph_app.astream(
        {"query": query},
        config={"configurable": {"thread_id": "s1"}}
    ):
        for node_name, output in event.items():
            yield output  # async generator!
```

Interview Questions — Async:

- *"Async kyun use karte ho FastAPI mein?"* — I/O bound operations concurrent handle karo, ek request wait kare toh doosri process hoti rahe, throughput better
- *"await kya karta hai?"* — coroutine ko pause karta hai, event loop ko control deta hai doosra kaam karne ke liye, result milne pe resume
- *"asyncio.gather kab use karte ho?"* — multiple independent async operations parallel chalani hon — 3 LLM calls simultaneously
- *"Sync vs Async — kab kaunsa?"* — I/O bound = async (API calls, DB, file). CPU bound = sync ya multiprocessing

---

📌 TOPIC 10: Pydantic v2 — Deep Dive ⭐

Interview Importance: ⭐ ⭐ ⭐ ⭐ ⭐ (Critical) — FastAPI, LangGraph, LLM structured output

---

🔷 Pydantic Kya Hai

Definition (English):

> Pydantic is a data validation library that uses Python type hints to automatically validate, parse, and serialize data — ensuring that data conforms to the expected structure and types at runtime.

Hinglish:

- Data validation aur type checking ka library
- TypedDict + validation + serialization = Pydantic
- FastAPI request/response, LangGraph state, LLM structured output — sab jagah
- Java ka `@Valid` + Bean Validation ka Python equivalent

```python
```

```python
from pydantic import BaseModel, Field, field_validator, model_validator
from typing import Optional, List
from datetime import datetime

# ═══════════════════════════════════
# BASIC MODEL
# ═══════════════════════════════════

class UserProfile(BaseModel):
    name: str                           # required field
    age: int                            # required, must be int
    email: str                          # required
    city: str = "Unknown"               # optional — default value
    skills: List[str] = []              # optional — default empty list
    is_active: bool = True              # optional — default True
    score: Optional[float] = None       # optional — can be None

# ─── AUTOMATIC VALIDATION ───
# Valid data
user = UserProfile(
    name="Rahul",
    age=25,
    email="rahul@example.com",
    skills=["Python", "Java"]
)
print(user.name)     # "Rahul"
print(user.city)     # "Unknown" — default
print(user.model_dump())  # dict mein convert karo

# Type coercion — Pydantic smart hai
user2 = UserProfile(
    name="Priya",
    age="25",           # string "25" → int 25 automatically!
    email="priya@ex.com"
)
print(user2.age)  # 25 (int) — Pydantic ne convert kiya

# Invalid data → ValidationError
try:
    invalid_user = UserProfile(
        name="Test",
        age="not_a_number",    # ← int mein convert nahi ho sakta
        email="test@ex.com"
    )
except ValidationError as e:
    print(e.errors())
```

```
    # [{'loc': ('age',), 'msg': 'value is not a valid integer', ...}]

# ═══════════════════════════════════
# FIELD — Advanced Configuration
# ═══════════════════════════════════

class EvaluationResult(BaseModel):
    question_id: str = Field(
        description="Unique identifier for the question",
        min_length=1,
        max_length=100
    )
    faithfulness: float = Field(
        description="How faithful is the answer to context",
        ge=0.0,    # greater than or equal to 0
        le=1.0,    # less than or equal to 1
        default=0.0
    )
    answer_relevancy: float = Field(ge=0.0, le=1.0, default=0.0)
    context_precision: float = Field(ge=0.0, le=1.0, default=0.0)
    context_recall: float = Field(ge=0.0, le=1.0, default=0.0)

    tags: List[str] = Field(
        default=[],
        description="Evaluation tags",
        max_length=10  # list mein max 10 items
    )

    evaluated_at: datetime = Field(
        default_factory=datetime.now
        # default_factory = function call karo, value compute karo
        # datetime.now directly nahi de sakte — har instance ke liye fresh time chah
    )

# ═══════════════════════════════════
# VALIDATORS — Custom Validation
# ═══════════════════════════════════

class LLMConfig(BaseModel):
    model: str
    temperature: float = Field(ge=0.0, le=2.0)
    max_tokens: int = Field(ge=1, le=128000)
    api_key: Optional[str] = None

    # @field_validator — specific field validate karo
    @field_validator("model")
    @classmethod
```

```python
    def validate_model_name(cls, v: str) -> str:
        """Model name valid hai?"""
        valid_models = [
            "gpt-4o", "gpt-4o-mini", "gpt-4-turbo",
            "claude-3-5-sonnet", "claude-3-haiku"
        ]
        if v not in valid_models:
            raise ValueError(f"Invalid model '{v}'. Valid: {valid_models}")
        return v

    @field_validator("api_key")
    @classmethod
    def validate_api_key(cls, v: Optional[str]) -> Optional[str]:
        """API key format check karo"""
        if v is not None and not v.startswith("sk-"):
            raise ValueError("API key must start with 'sk-'")
        return v

    # @model_validator — multiple fields ek saath validate karo
    @model_validator(mode="after")
    def validate_config_consistency(self):
        """
        mode="after" = sab fields validate hone ke baad run karo
        Cross-field validation ke liye
        """
        if self.model == "gpt-4o-mini" and self.max_tokens > 16000:
            raise ValueError(
                "gpt-4o-mini max_tokens 16000 se zyada nahi ho sakta"
            )
        return self

# Test karo
try:
    config = LLMConfig(
        model="invalid-model",
        temperature=0.7,
        max_tokens=1000
    )
except ValidationError as e:
    print(e.errors()[0]["msg"])
    # "Invalid model 'invalid-model'. Valid: [...]"

# ═══════════════════════════════════
# LLM STRUCTURED OUTPUT — Main Use Case
# ═══════════════════════════════════

class RAGEvaluation(BaseModel):
```

```python
    """LLM se structured evaluation results lena"""

    overall_score: float = Field(
        ge=0.0, le=1.0,
        description="Overall quality score 0-1"
    )
    verdict: str = Field(
        description="PASS ya FAIL"
    )
    strengths: List[str] = Field(
        description="Answer ki achhaiyan — list of points",
        min_length=1
    )
    weaknesses: List[str] = Field(
        description="Improvement areas",
        default=[]
    )
    hallucination_detected: bool = Field(
        description="Kya answer mein hallucination hai?"
    )
    improvement_suggestion: str = Field(
        description="Ek concrete improvement suggestion"
    )

    @field_validator("verdict")
    @classmethod
    def validate_verdict(cls, v: str) -> str:
        if v not in ["PASS", "FAIL"]:
            raise ValueError("verdict must be 'PASS' or 'FAIL'")
        return v

# LLM se structured output lo
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", temperature=0)
structured_llm = llm.with_structured_output(RAGEvaluation)

evaluation = structured_llm.invoke(f"""
Evaluate this RAG response:
Question: {question}
Context: {context}
Answer: {answer}

Provide a structured evaluation.
""")

# Ab evaluation ek RAGEvaluation object hai — type safe!
```

```python
print(evaluation.overall_score)         # 0.85
print(evaluation.verdict)               # "PASS"
print(evaluation.hallucination_detected) # False
print(evaluation.strengths)             # ["Accurate", "Complete"]

# ═══════════════════════════════════
# PYDANTIC IN LANGGRAPH STATE
# ═══════════════════════════════════

from pydantic import BaseModel
from langgraph.graph import StateGraph

# Pydantic model as LangGraph state (v2 approach)
class AgentState(BaseModel):
    messages: List[Any] = []
    query: str = ""
    docs: List[str] = []
    answer: str = ""
    error: Optional[str] = None
    iteration_count: int = 0

    class Config:
        arbitrary_types_allowed = True  # LangChain Message types ke liye

# ═══════════════════════════════════
# SERIALIZATION — dict aur JSON
# ═══════════════════════════════════

user = UserProfile(name="Rahul", age=25, email="r@ex.com")

# Dict mein convert karo
user_dict = user.model_dump()
# {"name": "Rahul", "age": 25, "email": "r@ex.com", ...}

# JSON string mein convert karo
user_json = user.model_dump_json()
# '{"name":"Rahul","age":25,...}'

# Dict se Pydantic model banao
data = {"name": "Priya", "age": 30, "email": "p@ex.com"}
user_from_dict = UserProfile(**data)
# ya
user_from_dict = UserProfile.model_validate(data)

# JSON se model banao
user_from_json = UserProfile.model_validate_json('{"name":"Amit",...}')
```

Interview Questions — Pydantic:

- *"Pydantic TypedDict se better kyun hai?"* — automatic validation, type coercion, Field constraints, validators, serialization built-in
- *"@field_validator kya karta hai?"* — specific field pe custom validation logic add karo
- *"LLM structured output mein Pydantic kaise use karte ho?"* — `.with_structured_output(PydanticModel)` — LLM guaranteed format mein return karta hai
- *"model_dump() kab use karte ho?"* — Pydantic object ko dict mein convert karna ho — API response, logging, storage

---

📌 TOPIC 11: TypedDict aur Type Hints

Interview Importance: ⭐ ⭐ ⭐ ⭐ (High) — LangGraph State ka core

---

```python
```

```python
from typing import (
    TypedDict, Annotated, Optional, Union,
    List, Dict, Tuple, Any, Literal, Callable
)

# ═══════════════════════════════════
# TYPE HINTS — Basic
# ═══════════════════════════════════

# Basic types
name: str = "Rahul"
age: int = 25
score: float = 0.85
active: bool = True

# Collection types
names: List[str] = ["Rahul", "Priya"]
config: Dict[str, Any] = {"model": "gpt-4o", "temp": 0.7}
coords: Tuple[float, float] = (28.6, 77.2)

# Optional — None ho sakta hai ya nahi
email: Optional[str] = None          # str ya None
phone: Optional[str] = "9999999999"  # dono valid

# Union — multiple types allowed
id_field: Union[str, int] = "abc123"  # string ya int dono
id_field = 12345                       # ab int — valid!

# Literal — specific values sirf
model: Literal["gpt-4o", "gpt-4o-mini", "claude"] = "gpt-4o"
status: Literal["active", "inactive", "pending"] = "active"

# Callable — function type
evaluator: Callable[[str, str], float]
# Callable[[arg_types], return_type]
# Yeh function ek function argument expect karta hai

# ═══════════════════════════════════
# TYPEDDICT — LangGraph State
# ═══════════════════════════════════

"""
TypedDict = dict with defined structure aur type hints
Dictionary jaisa use hota hai but typed
Java ka Map<String, Object> but with structure
"""
```

```python
class RAGState(TypedDict):
    # Required fields — default nahi hai
    messages: Annotated[List[Any], add_messages]
    query: str

    # Optional fields — total=False se
    docs: List[str]
    answer: str
    error: Optional[str]
    iteration_count: int

# TypedDict vs Pydantic:
# TypedDict = sirf type hints — runtime validation nahi
# Pydantic = type hints + runtime validation + serialization
# LangGraph State ke liye TypedDict preferred hai

# Use karo — dict ki tarah
state: RAGState = {
    "messages": [],
    "query": "RAG kya hai?",
    "docs": [],
    "answer": "",
    "error": None,
    "iteration_count": 0
}

# Access
print(state["query"])          # "RAG kya hai?"
state["iteration_count"] += 1  # update karo

# ═══════════════════════════════
# ANNOTATED — Extra Metadata
# ═══════════════════════════════

from typing import Annotated
from langgraph.graph.message import add_messages

# Annotated[type, metadata]
# metadata = koi bhi extra information — framework use karta hai

class AgentState(TypedDict):
    # add_messages = reducer — append karo, replace mat karo
    messages: Annotated[List[Any], add_messages]

    # Simple fields — reducer nahi chahiye
    query: str
```

```python
    answer: str

# Pydantic mein Annotated:
from pydantic import Field
class MyModel(BaseModel):
    age: Annotated[int, Field(ge=0, le=150)]
    # Annotated[int, Field(ge=0, le=150)] = int + validation constraint
```

📌 TOPIC 12: OOP — Classes aur Objects

Interview Importance: ⭐ ⭐ ⭐ (Medium) — Basic understanding — deep dive nahi

---

```python
```

```python
# ═══════════════════════════════════
# CLASS BASICS
# ═══════════════════════════════════

class EmbeddingService:
    """Embedding service class"""

    # Class variable — sab instances share karte hain
    supported_models = ["ada-002", "3-small", "3-large"]
    call_count = 0  # kitni baar call hua

    def __init__(self, model: str, api_key: str):
        """
        Constructor — Java ka constructor jaisa
        self = this (Java ka this)
        """
        # Instance variables — har instance ka apna
        self.model = model
        self._api_key = api_key       # _ = private convention
        self.__cache = {}             # __ = name mangling (truly private-ish)
        self.initialized_at = datetime.now()

    def embed(self, text: str) -> List[float]:
        """Public method"""
        EmbeddingService.call_count += 1  # class variable update

        # Cache check karo
        if text in self.__cache:
            return self.__cache[text]

        # Actual embedding logic
        result = self._call_api(text)
        self.__cache[text] = result
        return result

    def _call_api(self, text: str) -> List[float]:
        """Protected method — convention se private"""
        # API call logic
        pass

    @staticmethod
    def is_valid_model(model_name: str) -> bool:
        """Static — self nahi chahiye, class pe bhi call kar sako"""
        return model_name in EmbeddingService.supported_models

    @classmethod
```

```python
    def from_env(cls):
        """Class method — factory pattern"""
        import os
        return cls(
            model=os.getenv("EMBEDDING_MODEL", "3-small"),
            api_key=os.getenv("OPENAI_API_KEY", "")
        )

    def __repr__(self) -> str:
        """Object ka string representation — debugging ke liye"""
        return f"EmbeddingService(model={self.model})"

    def __len__(self) -> int:
        """len() call pe yeh run hoga"""
        return len(self.__cache)

# ═══════════════════════════════════
# INHERITANCE
# ═══════════════════════════════════

class BaseEvaluator:
    """Abstract base class"""

    def evaluate(self, question: str, answer: str, context: str) -> float:
        """Subclass mein implement karna padega"""
        raise NotImplementedError("Subclass must implement evaluate()")

    def batch_evaluate(self, test_cases: list) -> list:
        """Template method — subclass ka evaluate() use karta hai"""
        return [
            self.evaluate(tc["question"], tc["answer"], tc["context"])
            for tc in test_cases
        ]

class FaithfulnessEvaluator(BaseEvaluator):
    """Faithfulness check karta hai"""

    def __init__(self, threshold: float = 0.7):
        self.threshold = threshold
        super().__init__()  # Java ka super() jaisa

    def evaluate(self, question: str, answer: str, context: str) -> float:
        # Faithfulness calculate karo
        claims = self._extract_claims(answer)
        verified = [c for c in claims if c in context]
        return len(verified) / len(claims) if claims else 0.0
```

```python
    def _extract_claims(self, answer: str) -> list:
        # Claims extract karo
        pass

class RelevancyEvaluator(BaseEvaluator):
    """Answer relevancy check karta hai"""

    def evaluate(self, question: str, answer: str, context: str) -> float:
        # Relevancy calculate karo
        pass

# Multiple evaluators ek saath
evaluators = [FaithfulnessEvaluator(0.8), RelevancyEvaluator()]
for evaluator in evaluators:
    score = evaluator.evaluate(q, a, ctx)  # polymorphism!
```

📌 TOPIC 13: map(), filter(), zip(), enumerate()

Interview Importance: ⭐ ⭐ ⭐ (Medium) — Code mein dikhte hain — samajhna zaroori

---

```python
```

```python
# ═══════════════════════════════════
# map() — har item pe function apply karo
# ═══════════════════════════════════
numbers = [1, 2, 3, 4, 5]

# map(function, iterable) → iterator
doubled = list(map(lambda x: x * 2, numbers))
# [2, 4, 6, 8, 10]

# List comprehension se same kaam (preferred):
doubled = [x * 2 for x in numbers]  # ← yeh zyada readable hai

# AI use case
doc_texts = list(map(lambda doc: doc.page_content, retrieved_docs))
# vs
doc_texts = [doc.page_content for doc in retrieved_docs]  # better

# ═══════════════════════════════════
# filter() — condition pe filter karo
# ═══════════════════════════════════
scores = [0.9, 0.4, 0.7, 0.3, 0.8, 0.5]

# filter(function, iterable) → iterator
high_scores = list(filter(lambda s: s > 0.7, scores))
# [0.9, 0.8]

# Comprehension preferred:
high_scores = [s for s in scores if s > 0.7]  # clearer!

# ═══════════════════════════════════
# zip() — do lists ek saath iterate
# ═══════════════════════════════════
questions = ["Q1: RAG?", "Q2: LangGraph?", "Q3: RAGAS?"]
answers = ["RAG = ...", "LangGraph = ...", "RAGAS = ..."]
scores = [0.9, 0.8, 0.7]

# zip — parallel iteration
for q, a, s in zip(questions, answers, scores):
    print(f"Score {s}: {q[:20]} → {a[:20]}")

# Dict banana — zip se
eval_dict = dict(zip(questions, scores))
# {"Q1: RAG?": 0.9, "Q2: LangGraph?": 0.8, "Q3: RAGAS?": 0.7}

# ═══════════════════════════════════
# enumerate() — index + value
```

```python
# ═══════════════════════════════════
docs = ["Doc about RAG", "Doc about LangGraph", "Doc about RAGAS"]

for i, doc in enumerate(docs):
    print(f"Document {i+1}: {doc}")
# Document 1: Doc about RAG
# Document 2: Doc about LangGraph

# Start index custom karo
for i, doc in enumerate(docs, start=1):  # 1 se shuru
    print(f"[{i}] {doc}")

# AI use case — numbered citations
citations = "\n".join(
    f"[{i+1}] {doc.metadata['source']}"
    for i, doc in enumerate(retrieved_docs)
)
```

📌 TOPIC 14: String Formatting aur f-strings

Interview Importance: ⭐ ⭐ ⭐ (Medium) — Prompt templates mein critical

---

```python
```

```python
# ═══════════════════════════════════
# f-STRING — Most Important
# ═══════════════════════════════════
name = "Rahul"
score = 0.856789
docs_count = 5

# Basic
message = f"Hello {name}!"

# Expression andar
calc = f"2 + 2 = {2 + 2}"

# Method call andar
upper = f"{name.upper()} is great"

# Format specifiers
score_formatted = f"Score: {score:.2f}"    # 2 decimal places: "Score: 0.86"
score_percent = f"Score: {score:.1%}"      # percentage: "Score: 85.7%"
large_num = f"Count: {1000000:,}"          # comma separator: "Count: 1,000,000"
padded = f"{name:>10}"                     # right align 10 chars: "     Rahul"

# Multi-line f-string — PROMPT TEMPLATE
system_prompt = f"""You are an expert AI assistant.
Context provided: {docs_count} documents
User name: {name}
Quality threshold: {score:.0%}

Instructions:
1. Answer based ONLY on provided context
2. If unsure, say "I don't know"
3. Be concise and accurate"""

# Nested f-string
items = ["Python", "LangGraph", "FastAPI"]
formatted_list = f"Skills: {', '.join(items)}"
# "Skills: Python, LangGraph, FastAPI"

# ═══════════════════════════════════
# STRING METHODS — AI Mein Use
# ═══════════════════════════════════

raw_answer = "  The answer is: RAG retrieves relevant docs.  \n"

# Cleaning
cleaned = raw_answer.strip()          # whitespace remove — dono ends
```

```python
cleaned = raw_answer.strip().lower()  # lowercase bhi

# Check
if "RAG" in cleaned:
    print("RAG mentioned")

if cleaned.startswith("the"):
    print("Starts with 'the'")

# Split aur Join
sentences = cleaned.split(".")        # sentences mein tod do
rejoined = " ".join(sentences)        # wapas jodo

# Replace
formatted = cleaned.replace("RAG", "Retrieval-Augmented Generation")

# Template-style format
template = "Question: {q}\nAnswer: {a}\nScore: {s}"
filled = template.format(q="What is RAG?", a="RAG is...", s=0.9)
```

📌 BONUS: Python vs Java — Quick Reference

AI Engineer interviews mein yeh differences poochhe jaate hain

---

```
CONCEPT          JAVA                      PYTHON
──────────────────────────────────────────────────────
Null             null                      None
Boolean          true/false                True/False
Print            System.out.println()      print()
String format    String.format()           f-string
Array            int[] arr                 list
HashMap          HashMap<K,V>              dict {}
HashSet          HashSet<T>                set {}
For-each         for(T x : list)          for x in list
Try-finally      try-with-resources       with statement (context mgr)
Annotations      @Override, @NotNull      Decorators (@property, @tool)
Generics         List<String>             List[str] (type hints)
Optional         Optional<T>              Optional[str]
Interface        interface                 Abstract class / Protocol
Lambda           x -> x * 2               lambda x: x * 2
Stream           .stream().map().filter    List comprehension
Static method    static void method()     @staticmethod
Async            CompletableFuture        async/await
Null check       if (obj != null)         if obj: (truthy check)
String multi     "a" + "b" + "c"         f"{a}{b}{c}" or "".join()
```

📌 INTERVIEW QUESTIONS BANK

5.5 YOE AI Engineer — Most Asked (Product/MNC Companies India)

🟣 Python Basics

- **Q: Python mein mutable aur immutable kya hota hai? Examples do.**
  - A: Mutable = create hone ke baad change ho sakta hai — list, dict, set. Immutable = change nahi ho sakta — str, int, float, tuple. Why matters: Function arguments — mutable pass karo toh function change kar sakta hai (side effect). Immutable safe hai — function change nahi kar sakta.
- **Q: Python dynamic typing kya hai? Java se kaise different?**
  - A: Dynamic typing = type runtime pe decide hoti hai, pehle declare nahi karna. `x = 5` (int), phir `x = "hello"` (str) — valid. Java mein `int x = 5`, phir `x = "hello"` — compile error. Fayda: faster development. Nuksan: runtime type errors, IDE support kam.
- **Q: `is` vs `==` mein kya difference hai?**

- A: `==` = value same hai. `is` = same object hai memory mein (same id). `a = [1,2]`, `b = [1,2]`: `a == b` True (same value), `a is b` False (different objects). `None` check hamesha `is None` se karo — `== None` nahi.

- **Q: *args aur **kwargs explain karo.**
  - A: *args = variable positional arguments — tuple mein aate hain. **kwargs = variable keyword arguments — dict mein aate hain. Use karo jab function kitne arguments lega — woh pata nahi ho. LangChain functions mein common hai.

Intermediate

- **Q: Decorator kya hota hai? Apna example banao.**
  - A: Function jo doosra function wrap kare — extra behavior add kare. `@timer` decorator function execution time measure karta hai. Andar se: `my_func = timer(my_func)`. `@wraps` use karo original function ka naam preserve karne ke liye. AI mein: `@tool` LangChain tools ke liye, `@retry` rate limit handling ke liye, `@property` getters ke liye.

- **Q: Generator aur normal function mein kya fark hai?**
  - A: Normal function = ek baar mein sab return. Generator = `yield` se ek ek value lazily. Generator memory efficient hai — sab values RAM mein nahi. LLM streaming ke liye perfect — tokens ek ek generate hote hain, sab ek saath nahi aate. `for x in generator()` se iterate karo.

- **Q: Context manager kya hai? `with` statement kaise kaam karta hai?**
  - A: `__enter__` (setup) aur `__exit__` (cleanup) define karo. `with` statement ensure karta hai cleanup hamesha hoga — error aaye tab bhi. Java ka try-with-resources. AI mein: LangSmith tracing, DB connections, OpenAI cost tracking, MCP client.

- **Q: List comprehension vs map/filter — kaunsa better?**
  - A: List comprehension Python mein preferred hai — zyada readable aur Pythonic. `[x*2 for x in nums]` vs `list(map(lambda x: x*2, nums))`. Performance similar. Generator expression use karo memory ke liye: `(x*2 for x in nums)`.

AI Engineer Specific

- **Q: FastAPI mein async endpoint kyun banate ho?**
  - A: I/O bound operations — LLM API call, vector DB search, external APIs — sab network wait karte hain. Async mein ek request wait kare toh dusri process hoti hai —

event loop switch karta hai. Throughput better hota hai. CPU bound ke liye — `run_in_executor` ya background tasks.

- **Q: Pydantic structured output LLM ke saath kaise use karte ho?**
  - A: Pydantic model define karo expected output structure se. `llm.with_structured_output(MyModel)` — LLM guaranteed format mein dega. Free text parse karna fragile hai production mein — Pydantic se type safe aur validated output milta hai. Field validators add karo edge cases ke liye.
- **Q: @tool decorator andar se kya karta hai?**
  - A: Function naam → tool naam. Docstring → tool description (LLM yeh padhta hai decide karne ke liye). Type hints → JSON schema generate karta hai (kaunse args chahiye, kaunsi types). Function ko Tool object mein wrap karta hai. Isliye docstring bahut important hai — poor docstring = LLM wrong tool choose karta hai.
- **Q: Type hints sirf documentation ke liye hain ya runtime bhi?**
  - A: Pure Python mein sirf hints — runtime check nahi. But Pydantic + FastAPI runtime validation karte hain type hints se. LangGraph StateGraph type hints use karta hai structure ke liye. `mypy` tool static type checking karta hai. Production mein Pydantic use karo actual validation ke liye.

🔴 Advanced (Senior Level)

- **Q: GIL kya hai aur AI applications pe kaise affect karta hai?**
  - A: Global Interpreter Lock — ek waqt sirf ek Python thread truly execute hoti hai. CPU bound parallel execution limit hoti hai. But: I/O bound operations (LLM calls, DB queries) GIL release karte hain — async wahan kaam karta hai. CPU bound ke liye: `multiprocessing` module (alag process = alag GIL) ya NumPy/PyTorch jo C mein GIL release karta hai.
- **Q: Python memory management kaise kaam karta hai?**
  - A: Reference counting — har object kitne references hain count karta hai. Zero references → garbage collected. Circular references ke liye cyclic garbage collector bhi hai. `del` explicit delete karta hai reference. Large objects ko explicitly `del` karo memory-sensitive applications mein — embedding batches, large document lists.
- **Q: Coroutine aur Thread mein kya difference hai?**
  - A: Thread = OS-level, truly parallel (GIL limit). Coroutine = single thread mein cooperative multitasking — `await` pe voluntarily switch. Threads: overhead zyada, memory zyada, race conditions. Coroutines: lightweight, no race conditions (single thread), perfect for I/O bound. LLM apps mostly I/O bound → async/await preferred.

📌 QUICK REVISION CHEATSHEET

```
DATA TYPES
├── str: immutable, f-string use karo
├── list: mutable, ordered, duplicates ok
├── dict: key-value, ordered (3.7+)
├── tuple: immutable list, multiple return values
└── set: unique values, fast lookup

FUNCTIONS
├── *args = variable positional → tuple
├── **kwargs = variable keyword → dict
├── lambda = anonymous one-liner function
└── first-class = functions = objects = pass around

LIST COMPREHENSION
├── [expr for x in iterable]
├── [expr for x in iterable if condition]
├── {k: v for k, v in dict.items()} — dict comp
└── (expr for x in iterable) — generator expression (lazy!)

DECORATOR
├── @decorator = func = decorator(func)
├── @wraps — original naam preserve karo
├── @property — getter/setter clean syntax
├── @staticmethod — self nahi chahiye
├── @classmethod — cls milta hai
└── @tool — LangChain tool banana

GENERATOR
├── yield — ek ek value, lazy
├── Memory efficient — sab ek saath nahi
└── LLM streaming ke liye perfect

CONTEXT MANAGER
├── with statement — setup + cleanup automatic
├── __enter__ = setup, __exit__ = cleanup
├── @contextmanager — simpler way
└── LangSmith, DB, file handling mein use

ASYNC
├── async def — coroutine define karo
├── await — pause, event loop switch karo
├── asyncio.gather() — parallel execution
```

```
└── FastAPI: async endpoint = better throughput

PYDANTIC
├── BaseModel — data validation
├── Field() — constraints, description
├── @field_validator — custom validation
├── @model_validator — cross-field validation
└── .with_structured_output() — LLM structured output

TYPE HINTS
├── Optional[str] — str ya None
├── List[str] — list of strings
├── Dict[str, Any] — dict
├── Literal["a", "b"] — specific values
└── Annotated[type, metadata] — type + extra info
```

Notes by: Python Complete Guide | AI Engineer Interview Prep Yeh guide cover karta hai: Basics → Decorators → Generators → Async → Pydantic Next: Week 3 aur Week 4 ke notes padho — LLM Evaluations + System Design